pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Case Study: Joining the Tables

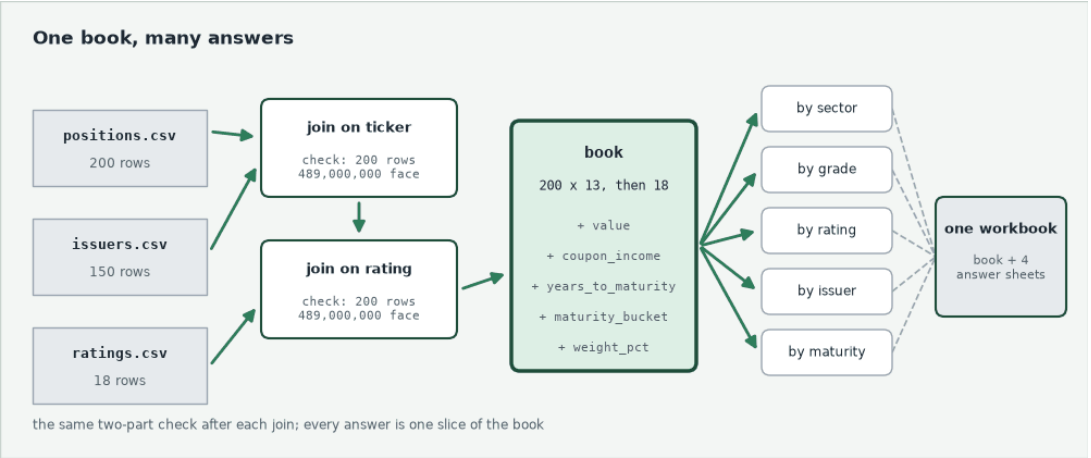
WEEK 2, DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's fifteenth note

Yesterday's files passed every check. Join them into one book and hold it to your numbers: 200 rows, 13 columns, 489,000,000 of face. Give every bond a value at the clean price, its coupon income, and a weight. Then where does the money sit: by sector, by grade (by count and by value, and say which), by rating in credit order? What is our average rating, which the letters could not give you? Which ten issuers carry the most, and what share? What matures first, and who are yesterday's seven flagged bonds? Send one workbook, the book and the answers, and leave nothing behind.



File	One row is	Rows, columns	Joins on
`positions.csv`	one bond held, 489,000,000 of face in all	200, 6	`ticker`
`issuers.csv`	one issuer, every ticker found	150, 5	`ticker, then rating`
`ratings.csv`	one rating, every rating found	18, 4	`rating`

Prices are clean, per 100 of face; coupons are annual percentages of face. "maturity" arrives as text.

One book, then one line per question.

10.1 Two numbers stored, then the first join

```
rows_before = positions.shape[0]
face_before = positions['par_held'].sum()
print(rows_before, face_before)
200 489000000
```

```
print(step_one[['issuer', 'sector', 'rating']].isna().sum())
issuer    0
sector    0
rating    0
dtype: int64
```

```
step_one = pd.merge(positions, issuers, on='ticker', how='left', validate='many_to_one')
assert step_one.shape[0] == rows_before, 'the join changed the row count'
assert step_one['par_held'].sum() == face_before, 'the join changed the face'
print(step_one.shape)
(200, 10)
```

Ten columns, as yesterday's Predict 5 wrote down; both "assert" lines were silent. "==" is exact, right for a count and for whole dollars; decimals will need NumPy Part 1's "np.isclose()".

No gaps. BEZE reads Bezane Finance, Financials, BBB-; both BEZL bonds carry one issuer row: many to one.

Rows and face unchanged: the first join is safe.

No grade until the second join, then the split ties

```
step_one.groupby('grade')['par_held'].sum()
KeyError: 'grade'
```

The grade is in the third table.

```
book = pd.merge(step_one, ratings, on='rating', how='left', validate='many_to_one')
assert book.shape[0] == rows_before, 'the join changed the row count'
assert book['par_held'].sum() == face_before, 'the join changed the face'
print(book.shape, book[['score', 'bucket', 'grade']].isna().sum().sum())
book.iloc[0]
(200, 13) 0
```

```
cusip               99080CAE8
ticker                   BEZE
coupon                    7.0
maturity           2032-06-18
price                 106.727
par_held              5000000
issuer         Bezane Finance
sector              Financials
country                     US
rating                   BBB-
score                      10
bucket                    BBB
grade       Investment Grade
Name: 0, dtype: object
```

```
book.groupby('grade')['par_held'].sum()
grade
High Yield          169250000
Investment Grade    319750000
Name: par_held, dtype: int64
```

One row is still one bond held, now carrying its issuer and its grade. 169,250,000 plus 319,750,000 is 489,000,000.

200 rows, 13 columns, 489,000,000: yesterday's numbers hold.

10.2 Keep everything, and the row check stops it

```
everything = pd.merge(positions, issuers, on='ticker', how='outer', indicator=True)
print(everything.shape)
print(everything['_merge'].value_counts())
everything[everything['_merge'] == 'right_only'].head(1)
(235, 11)
_merge
both          200
right_only     35
left_only       0
Name: count, dtype: int64
```

```
rows_after = everything.shape[0]
assert rows_after == rows_before, f'expected {rows_before} rows, found {rows_after}'
AssertionError: expected 200 rows, found 235
```

The 35 "right_only" rows are yesterday's issuers with no bond: "NaN" where the bond should be.

Left join, then both checks: rows and face.

The join written as	Rows	Face	Caught by
`how='left', validate='many_to_one'`	200	489,000,000	passes both checks
`how='inner'`	200 here	489,000,000 here	both checks, the month a ticker goes missing
`a ticker listed twice in issuers`	more	more	validate= refuses it before either check

Excel side by side. A lookup column in G

Job	pandas	Excel
A ticker not found	`NaN, counted with isna().sum()`	#N/A, counted with =SUMPRODUCT(--ISNA(G2:G201))
A ticker listed twice	`rows repeated; validate= refuses`	VLOOKUP, and XLOOKUP in Excel 2021 and Microsoft 365, take the first match, silently

10.3 Value, coupon income, and a weight for every bond

```
book['value'] = book['par_held'] * book['price'] / 100
book[['ticker', 'par_held', 'price', 'value']].head(3)
```

	ticker	par_held	price	value
0	BEZE	5000000	106.727	5336350.0
1	BEZH	250000	97.556	243890.0
2	BEZL	1500000	98.730	1480950.0

BEZE's 5,000,000 at 106.727 is 5,336,350.

```
import numpy as np
assert np.isclose(book['weight_pct'].sum(), 100), 'the weights do not add to 100'
print(book['weight_pct'].sum(), book.shape)
100.0 (200, 18)
```

```
book['coupon_income'] = book['par_held'] * book['coupon'] / 100
print(book['coupon_income'].iloc[0], book['coupon_income'].sum())
350000.0 28864375.0
```

```
book['weight_pct'] = book['value'] / total_value * 100
book.nlargest(3, 'weight_pct')[['ticker', 'par_held', 'price', 'weight_pct']]
```

	ticker	par_held	price	weight_pct
0	BEZE	5000000	106.727	1.104779
59	FESA	5000000	106.471	1.102129
104	LORX	5000000	104.801	1.084842

Three 5,000,000 positions above par: weight follows value, not face.

Excel side by side. Coupon in C, maturity in D, price in E, face in F, value in N

Column	pandas	Excel, filled down
value	book['par_held'] * book['price'] / 100	=F2 * E2 / 100
coupon_income	book['par_held'] * book['coupon'] / 100	=F2 * C2 / 100
years_to_maturity	(book['maturity'] - as_of).dt.days / 365.25	=(D2 - DATE(2026, 9, 30)) / 365.25
weight_pct	book['value'] / total_value * 100	=N2 / SUM($N$2:$N$201) * 100, the anchors keep the total fixed

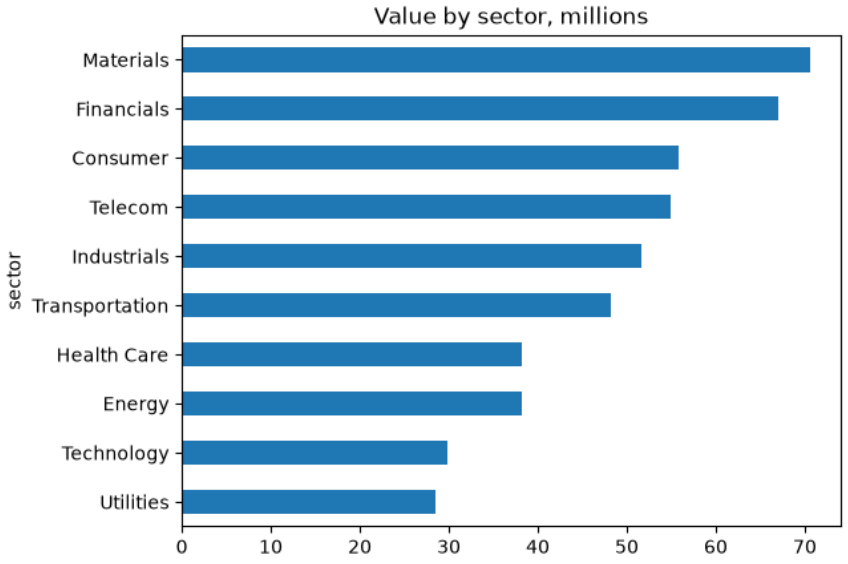
Every bond has a value, an income, and a weight that add up: 200 rows, 18 columns.

10.4 Value by sector, adding back to the whole

```
by_sector = book.groupby('sector').agg(bonds=('cusip', 'count'), value=('value', 'sum'))
by_sector['share_pct'] = by_sector['value'] / total_value * 100
by_sector = by_sector.sort_values('value', ascending=False)
assert by_sector['bonds'].sum() == rows_before, 'a bond is missing from the sectors'
assert np.isclose(by_sector['value'].sum(), total_value), 'value is missing from the sectors'
by_sector.round(2)
```

sector	bonds	value	share_pct
Materials	26	70527440.0	14.60
Financials	26	67067357.5	13.88
Consumer	26	55773490.0	11.55
Telecom	21	54953580.0	11.38
Industrials	20	51658930.0	10.69
Transportation	20	48159510.0	9.97
Health Care	20	38268482.5	7.92
Energy	17	38258305.0	7.92
Technology	12	29826430.0	6.17
Utilities	12	28530705.0	5.91

From here every answer describes the data; none is a view on a bond or a sector.



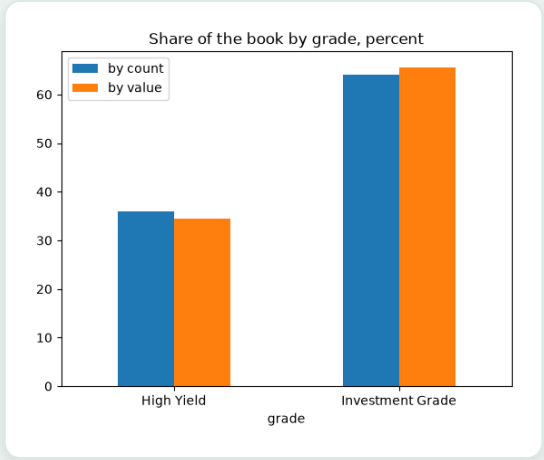
Materials leads at 14.60 percent and Utilities trails at 5.91; the top three each hold 26 bonds.

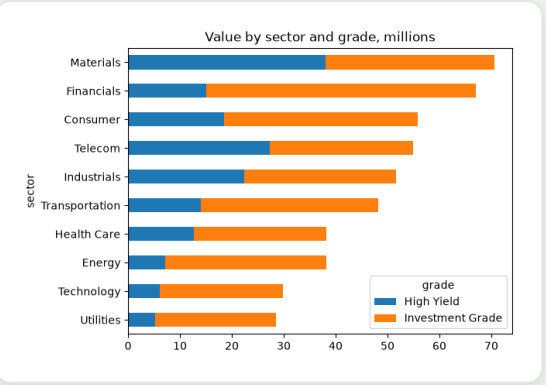
Excel side by side. A PivotTable with sector in Rows and Sum of value in Values; a second Sum of value, set to Show Values As, % of Grand Total, gives "share_pct".

By count or by value: say which a share is of

```
by_grade = book.groupby('grade').agg(bonds=('cusip', 'count'), value=('value', 'sum'))
shares = pd.DataFrame({'by count': by_grade['bonds'] / rows_before * 100,
                       'by value': by_grade['value'] / total_value * 100})
print(shares.round(2))
chart = shares.plot.bar(rot=0, title='Share of the book by grade, percent')
                 by count  by value
grade
High Yield           36.0     34.45
Investment Grade     64.0     65.55
```

Investment grade is 64.0 percent of the bonds and 65.55 percent of the value: say which a share is of.





Excel side by side

Grade in M, value in N: "=COUNTIF(M:M, "High Yield")" counts, "=SUMIF(M:M, "High Yield", N:N)" adds the value.

71 of the 128 investment grade bonds and 42 of the 72 high yield: each count met its own grade's, matched by label as in pandas Part 1. Below par compares a price with 100; it judges no bond.

Materials is the one sector where the high yield part of the bar is longer than the investment grade part, as Combining Part 2 found on market value; Telecom is near even. In Excel: Insert, Bar, Stacked Bar on the grid.

Every share says what it was divided by.

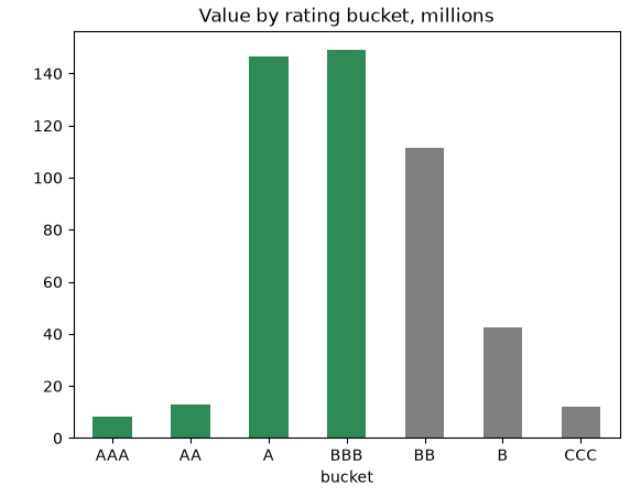
10.5 Credit order from the score, and an average rating

```
by_bucket = book.groupby('bucket').agg(first_score=('score', 'min'), bonds=('cusip', 'count'),
                                       value=('value', 'sum'))
by_bucket = by_bucket.sort_values('first_score')
by_bucket['share_pct'] = (by_bucket['value'] / total_value * 100).round(2)
by_bucket
```

"groupby" sorts text, so A would come before AA; Combining Part 1 typed "credit_order" by hand. This order comes from the scale itself, so next year's file needs no new list.

```
weighted_score = (book['score'] * book['value']).sum() / total_value
weighted_coupon = (book['coupon'] * book['value']).sum() / total_value
print('Score, simple:', round(book['score'].mean(), 2), 'weighted:', round(weighted_score, 2))
print('Coupon, simple:', book['coupon'].mean(), 'weighted:', round(weighted_coupon, 4))
Score, simple: 9.36 weighted: 9.25
Coupon, simple: 5.9375 weighted: 5.9403
```

9.25 sits between BBB, score 9, and BBB-, score 10: the average rating yesterday's "TypeError" refused. It treats every notch as one equal step, a convention, not a fact.



BBB and A hold 30.82 and 30.38 percent; the three grey bars add to 34.45, the high yield share.

Excel side by side

Within a grade the divisor is that grade's value, not the book's. One grade, with coupon in C, grade in M, value in N: "=SUMPRODUCT((M2:M201="High Yield") * C2:C201 * N2:N201) / SUMIF(M2:M201, "High Yield", N2:N201)".

10.6 Two summaries, and yesterday's seven named

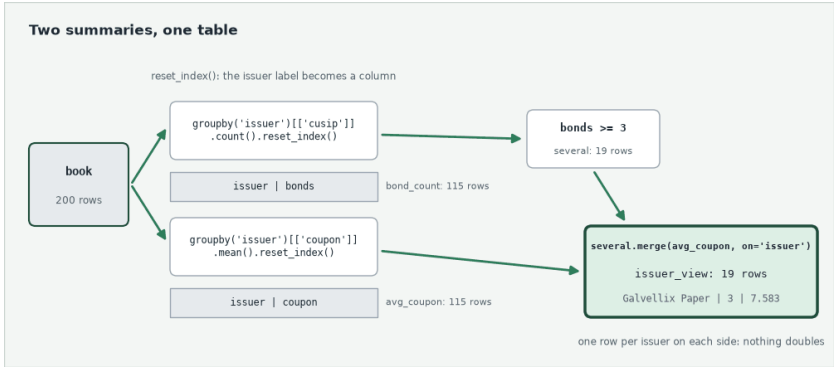
```
avg_coupon = book.groupby('issuer')[['coupon']].mean().reset_index()
bond_count = book.groupby('issuer')[['cusip']].count().reset_index()
bond_count = bond_count.rename(columns={'cusip': 'bonds'})
several = bond_count[bond_count['bonds'] >= 3]
issuer_view = several.merge(avg_coupon, on='issuer', how='left')
print(issuer_view.shape)
issuer_view.sort_values('coupon', ascending=False).head(3)
(19, 3)
```

	issuer	bonds	coupon
7	Galvellix Paper	3	7.583333
11	Krilellix Networks	3	6.416667
8	Grovostrin Freight	3	6.375000

Each table has one row per issuer, so the merge cannot double a row.

Excel side by side

One PivotTable, issuer in Rows: Count of cusip and Average of coupon in Values, then the Row Labels arrow, Value Filters, Greater Than Or Equal To, 3 on the count. With Sum of value in Values instead, Value Filters, Top 10 gives the ten largest issuers.



	ticker	issuer	sector	rating	grade
3	BEZL	Bezoquil Freight	Transportation	BB	High Yield
18	CLDB	Calderby Retail	Consumer	B	High Yield
69	GALR	Galventor Power	Utilities	AA+	Investment Grade
86	HULR	Hulvimer Pipeline	Energy	BBB	Investment Grade
137	QVNE	Quorvane Energy	Energy	BBB	Investment Grade
174	VRAR	Vraxentor Retail	Consumer	BBB+	Investment Grade
184	WYXA	Wyxundra Energy	Energy	B+	High Yield

The join puts a name on every number.

10.7, 10.8 What matures first, and one workbook

```
within_three = book['maturity'] <= pd.Timestamp('2029-09-30')
print(within_three.sum(), book.loc[within_three, 'par_held'].sum())
print(round(book.loc[within_three, 'weight_pct'].sum(), 2))
print(book.loc[within_three, 'grade'].value_counts())
28 74000000
15.29
grade
Investment Grade    17
High Yield          11
Name: count, dtype: int64
```

```
maturing = book[within_three].sort_values('maturity')
maturing[['ticker', 'issuer', 'maturity', 'rating', 'grade']].head(5)
```

	ticker	issuer	maturity	rating	grade
177	VYRH	Veyrholt Health	2027-10-02	BB	High Yield
112	NEVR	Nevovar Energy	2027-10-24	BBB-	Investment Grade
70	GALX	Galvellix Paper	2027-10-27	B-	High Yield
125	PLWK	Pellwick Paper	2027-10-28	A-	Investment Grade
56	FARX	Farvellix Paper	2027-12-22	BB+	High Yield

Excel side by side. A PivotTable with grade and bucket both in Rows, then grade dragged from Rows to Columns.

28 bonds, 15.29 percent of the book, mature first.

```
with pd.ExcelWriter('desk_book.xlsx') as writer:
    book.to_excel(writer, sheet_name='book', index=False)
    by_sector.to_excel(writer, sheet_name='sectors')
    by_grade.to_excel(writer, sheet_name='grades')
    by_bucket.to_excel(writer, sheet_name='buckets')
    top_ten.to_excel(writer, sheet_name='top_ten')
```

```
sheets = pd.read_excel('desk_book.xlsx', sheet_name=None, dtype={'cusip': str})
for name, table in sheets.items():
    print(name, table.shape)
book (200, 18)
sectors (10, 4)
grades (2, 3)
buckets (7, 5)
top_ten (10, 2)
```

```
back = sheets['book']
assert back['par_held'].sum() == face_before, 'the face changed in the file'
assert np.isclose(back['value'].sum(), total_value), 'the value changed in the file'
os.remove('desk_book.xlsx')
print(os.path.exists('desk_book.xlsx'))
False
```

The note is answered in one file; next month the same cells run again.

Predict first: decide, then run each in the notebook

Predict 1

"everything" has 235 rows, 35 of them with no face. What does "everything['par_held'].sum()" give? A) 489,000,000, the positions' face. B) More than 489,000,000. C) "NaN": the blanks spoil the total.

```
print(everything['par_held'].sum())
```

Predict 3

"weight_pct" already holds each bond's share of the value. What does the next cell print? Write down a number.

```
print(round((book['coupon'] * book['weight_pct']).sum(), 2))
```

Predict 2

The simple average price is 98.83, under par. The next cell adds up "value". Against the 489,000,000 of face, the total is: A) Above it. B) Below it. C) Equal: premiums and discounts cancel.

```
total_value = book['value'].sum()
print('Value:', total_value)
print('Per 100 of face:', round(total_value / face_before * 100, 3))
```

Predict first: the ten largest, and two labels

Predict 4

115 issuers stand behind the 200 bonds. What share of the book's value do the ten largest hold? A) Under 10 percent: ten is under a tenth of 115. B) About 20 percent. C) Over 40 percent.

```
by_issuer = book.groupby('issuer')['value'].sum()
top_ten = by_issuer.nlargest(10)
print(len(by_issuer), top_ten.sum(), round(top_ten.sum() / total_value * 100, 2))
chart = (top_ten / 1000000).sort_values().plot.barh(title='The ten largest issuers, millions')
```

Predict 5

"groupby()" also takes a list of two columns. How many rows does "book.groupby(['grade', 'maturity_bucket'], observed=True)['value'].sum()" return? Write down a number.

```
by_two = book.groupby(['grade', 'maturity_bucket'], observed=True)['value'].sum()
by_two
```

The note, answered

The note asked	Answer
One book	200 rows, 13 columns, 489,000,000 of face
By sector	Materials first, 14.60 percent
By grade	investment grade 64.0 percent by count, 65.55 by value
By rating	BBB 30.82 percent, A 30.38
Average rating	score 9.25, between BBB and BBB-
Maturing first	28 bonds, 15.29 percent, from 2027-10-02
Yesterday's seven	four investment grade, three high yield
One workbook	five sheets, read back, removed

Be proud: you ran week 2's whole toolkit, loading, checking, joining, summarizing, and saving, as one job the desk can rerun next month.

Next week opens with Visualization: the same book, drawn chart by chart with every part labelled.